

TETI LAB SKILL - TLS 2026

Nama: Muhammad Alif Haryadi

Kelompok: Onyx

PROBLEM 1: THE LAST ASTRONAUT

[PSEUDOCODE]:

PROGRAM TheLastAstronaut

DECLARE N, K, current_index, remaining_count, eliminated_num AS INTEGER

DECLARE astronauts AS ARRAY OF INTEGER

BEGIN

 GET N

 GET K

 // Populate the circle with astronaut IDs 1 through N

 FOR i FROM 0 TO N - 1 DO

 astronauts[i] = i + 1

 END FOR

 SET current_index = 0

 SET remaining_count = N

 PRINT "Elimination Order: "

 WHILE remaining_count > 1 DO

 // Calculate target index using zero-based modular arithmetic

 current_index = (current_index + K - 1) MOD remaining_count

 eliminated_num = astronauts[current_index]

 PRINT eliminated_num

 // Remove astronaut by left-shifting array elements

 FOR i FROM current_index TO remaining_count - 2 DO

 astronauts[i] = astronauts[i + 1]

 END FOR

 remaining_count = remaining_count - 1

```

// Modify step size K based on astronaut ID parity
IF eliminated_num MOD 2 == 0 THEN
    K = K + 2
ELSE
    K = K - 1
END IF

// Enforce minimum threshold rule
IF K < 2 THEN
    K = 2
END IF
END WHILE

PRINT "Last Astronaut Remaining: ", astronauts[0]
END

```

[LOGIC EXPLANATION]:

The core strategy for solving "The Last Astronaut" relies on maintaining a circular data structure simulation through zero-based modular arithmetic. Initially, an array of size N is populated with unique sequential values ranging from 1 to N representing the astronauts standing in a circle. Because counting resumes directly after an eliminated person, we track an active pointer index (`current_index`). By advancing the index by $(K - 1)$ steps and applying a modulo operation relative to the remaining population count, we pinpoint which astronaut receives the K -th count without needing physical circular pointers. Once identified, the eliminated astronaut is printed and removed by shifting subsequent array entries leftward.

After each elimination, the system updates the variable K based on the parity of the eliminated astronaut's identifier. If the eliminated astronaut's number is even, K increases by 2; if odd, K decreases by 1. To maintain valid program bounds, a conditional threshold check resets K to 2 if any decrease causes K to drop below 2. This iterative process repeats inside a WHILE loop until only 1 astronaut remains in the array. The survivor left at index 0 is then outputted as the final remaining astronaut.

PROBLEM 2: ALIEN-IN-THE-MIDDLE

[LOGIC EXPLANATION]:

The "Alien-In-The-Middle" cipher system transforms plain text messages by shifting each character dynamically based on the alphabetic value of the original character immediately preceding it. The first character of the message serves as the anchor point and undergoes no transformation. To enforce consistent letter mapping, all characters are converted to uppercase upon entry. For every subsequent character from index 1 onward, the algorithm captures its original letter value before applying the positional transformation.

To calculate the position shift, the preceding character's 1-based alphabetic order ($A=1, B=2, \dots, Z=26$) is computed as $(\text{previousOriginal} - A) + 1$. This value is added to the 0-based index of the current letter. Standard modulo 26 arithmetic wraps any values extending beyond Z back into the alphabet loop. After updating the character at the current index, the unencrypted character is preserved in a temporary state variable (`previousOriginal`) so that the subsequent letter can reference it. The iteration continues sequentially through the array until the entire string is encrypted.